Data Visualization - COM-480 - Milestone 2

PROJECT GOAL OVERVIEW

The objective of our project, *unCharted*, is to build an interactive visualization of the New Delhi metro system. Since the metro first opened in 2002, the network has grown significantly across multiple phases. Users can observe both its geographic spread and its structural connections until today, while also having access to specific details for each station, such as its history, layout type, and the lines it serves. The core of the project is an interactive map combined with a timeline animation. This setup links together three main dimensions of data:

- **Spatial:** We map the exact, real-world geographic coordinates of hundreds of stations across the region to show the physical reach and accessibility of the metro over time.
- **Topological:** We visualize how the network connects, specifically highlighting how different metro lines cross paths to create important transfer stations and passenger routes.
- **Temporal:** We use historical opening dates to animate the step-by-step growth of the infrastructure from the year 2002 to 2025.

By combining these three aspects, our goal is to go beyond standard, static transit maps. We want to deliver a narrative experience where users can watch the construction of the metro network unfold.

TOOLS AND TECHNOLOGIES

To implement our visualization idea, here is a list of the main tools we used or will use:

- Sketching: for brainstorming and getting primary idea of what we want to visualize (**Lecture: Design for data viz**)
- D3.js: for building interactive visualizations (particle flow, ridership heatmap), data-driven transformations, Timeline animations (**Lectures: D3.js; More interactive d3.js**)
- Web scraping: Scrapy in Python (**Lecture: Data**)
- Tweet map, Navigation, Zooming, Grafana plot exploration (**Lecture: Interactions**)
- JavaScript / HTML / CSS: for the web interface (timeline, station detail panel, legend, widgets, stats)
- Python (preprocessing): for data cleaning and merging datasets
- Wikipedia MediaWiki API (images/descriptions): Information panel data
- Build tool: Vite
- Styling: Custom CSS (glassmorphism, CSS variables)
- Interactive dark-themed map centered on Delhi with Leaflet + D3 SVG overlay (**Lectures: Perception, color; Maps; Maps in practice**)
- Stadia Maps API to fetch the dark-themed tiles, providing the core geographic data that Leaflet uses to visualize the map with our intended aesthetic.
- Metro lines with different colors, and stations with dots on them, all appearing or disappearing depending on their year of opening; Connection lines between neighbor stations on each line (**Lectures: Perception, color; Mark and channels**)

PROJECT BREAKDOWN AND MINIMUM VIABLE PRODUCT (MVP)

We have decided to divide the project into four distinct, very important, components. Our core MVP focuses on the successful integration of these modules to produce a bounded, interactive Leaflet map of Delhi that displays stations and lines accurately based on geographic coordinates, coupled with a temporal slider that filters the network's visibility by opening year. Here is the detailed breakdown of the work within each module :

1. Data Collection, Preprocessing, and Scraping (Python)

This part is the foundation of the visualization, ensuring our spatial and temporal data is accurate and correctly formatted for D3.js.

- **Data Cleaning & Integration:** We use Python to parse, clean, and merge raw CSV files containing station metadata (ID, name, line color, layout, opening year) and topological data (line stops and ordered connections).
- **Web Scraping Pipeline:** We are also utilizing python script to actively scrape missing geographic and historical data for approximately 40 undocumented stations to ensure network completeness.
- **External API Preparation:** We are structuring the data pipeline to easily accept dynamic calls to the Wikipedia MediaWiki API, which will be used to fetch live descriptions and images for the front-end modals.

2. Geographic Projection and Base Map Rendering (Leaflet & D3.js)

This part handles the spatial representation of the metro network (principally the map):

- **Base Map Configuration:** We initialized a bounded Leaflet map centered exclusively on Delhi, utilizing a custom dark-themed tile layer to ensure the brightly colored metro lines stand out clearly against the background.
- **SVG Overlay & Coordinate Mapping:** We attached a responsive SVG pane directly over the Leaflet map. As the user zooms or pans, a custom function recalculates real-world geographic coordinates (Latitude/Longitude) into accurate pixel layer points.
- **Data Binding (Nodes & Edges):** Using D3.js, we bind our preprocessed data arrays to SVG elements. Stations are rendered as interactive circles (nodes) with specific stroke colors and hover-glow effects, while the connections between them are rendered as accurate SVG lines (edges) colored according to their respective metro line.

3. Temporal Animation Logic

This part is to bring the static map to life, allowing users to explore the spatial growth of the network over a 23-year period (from 2002 to 2025).

- **State Filtering:** We implemented a central update loop tied to an HTML range slider. Whenever the slider value changes, the logic filters the master data arrays, ensuring that only stations and connections with an opened value less than or equal to the current year are passed to the rendering engine.

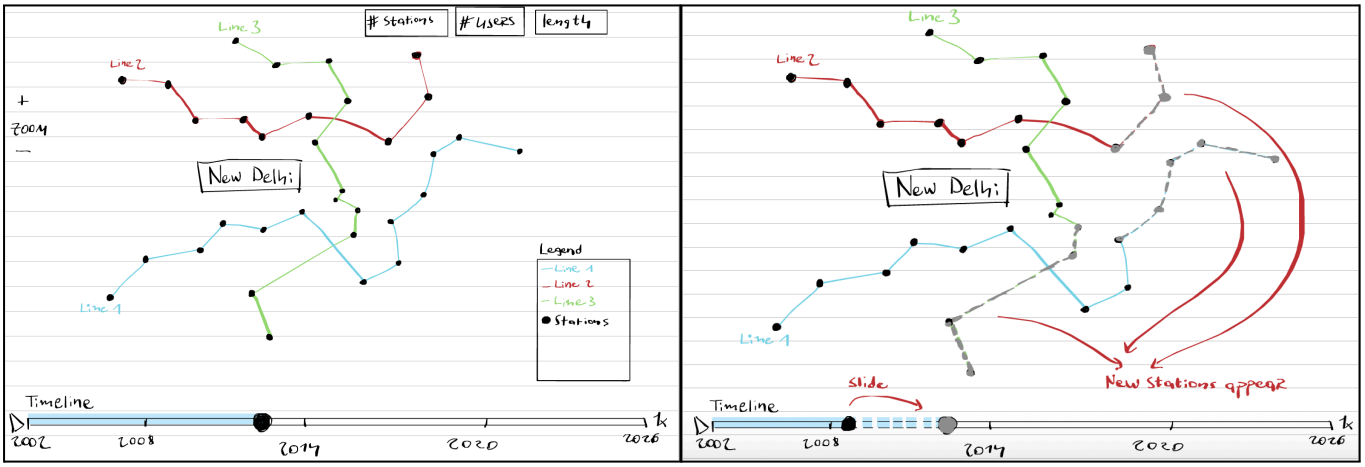
- **Automated Playback:** We built a JavaScript interval loop attached to a "Play/Pause" control, which programmatically increments the year and updates the timeline progress bar.
- **D3 Transitions:** To make the historical growth visually intuitive, we use D3 transition functions. When a station "opens" in the current active year, it appears using an elastic scale animation, while new connecting lines fade in smoothly using opacity transitions.

4. Interface Components (Widgets & Panels)

This module encompasses the surrounding UI elements that provide context, statistics, and specific details to the user.

- **Dynamic Statistics Header:** A reactive header that recalculates and displays network statistics on the fly. As the timeline progresses, JavaScript dynamically counts and updates the active number of stations and lines currently visible on the map.
- **Interactive Legend:** A dynamic legend that updates alongside the timeline, showing only the metro lines that are operational in the currently selected year, alongside the count of active stations per line.
- **Station Information Panels:** We implemented an event listener on the station nodes. When clicked, a hidden HTML modal slides into view, dynamically injected with the clicked station's specific metadata (name, layout type, opening year, and dynamically colored line badges). Future iterations of this module will embed data-driven ridership graphs into these panels.

Sketch 1 (Map and Network Structure) and Sketch 2 (Temporal Evolution)

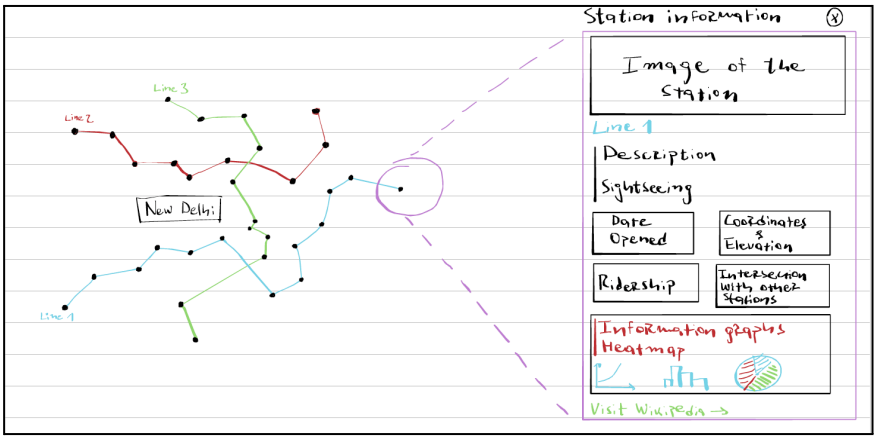


1) Map and Network Structure:
- A dark-themed map centered on Delhi using Leaflet
- Stations positioned using real geographic coordinates
- Stations are visualized as points and connections as lines. Metro lines encoded using different colors

2) Temporal Evolution:
- A timeline slider (2002–2025) with play/pause
- Stations and connections appear/disappear depending on station opening year
- Smooth transitions using D3 animations

EXTRA IDEAS *(If time permits)*: Our primary goal will be fully populating the currently empty station panels with dedicated, station-specific data and embedded graphs, alongside adding custom visual highlights on the map to clearly identify transfer stations. If further time allows, we will explore two advanced "bonus" features. First, we could generate interactive travel-time zones on the map, allowing users to select a station and see exactly how far they can travel across the network within a 15, 30, or 45-minute window. Second, we could transform the network map into a dynamic traffic flow visualization, where the connections between highly frequented stations dynamically become thicker or glow brighter, instantly highlighting the busiest transit corridors and bottlenecks across the city.

Sketch 3 (Station Information Panel): *Clickable stations where a panel opens with name, metro line, layout type, opening year, image (via Wikipedia API). Legend shows active metro lines and displays number of stations per line Zoom and navigation on the map (Lecture: Interactions)*



FUNCTIONAL PROTOTYPE REVIEW

Our functional prototype is actively running using Vite. We have built the main interface skeleton, which includes the dark-themed map container, interactive timeline controls, and statistics header; and successfully integrated Leaflet and D3.js to bind our parsed CSV data to the map. This allows the system to dynamically render the network's spatial evolution over time. Our immediate next steps are to strictly limit the map's navigation bounds to center exclusively around Delhi, expand our web scraping pipeline to retrieve missing data for approximately 40 stations, and fully implement the station information panel to display dedicated data, graphs and transfer-station highlights.

Link to the website : https://uncharted-delhi.netlify.app/